

TUGAS
SISTEM OPERASI



Disusun oleh :
Muhammad Rafi Hendardi
2410651062

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

BAB I

EXPERIMENT DENGAN DATA BERBEDA

Jalankan setiap algoritma dengan data berikut dan catat hasilnya:

Data Set 1:

- P1: AT=0, BT=10
- P2: AT=1, BT=5
- P3: AT=2, BT=8

Data Set 2:

- P1: AT=0, BT=5
- P2: AT=1, BT=3
- P3: AT=2, BT=8
- P4: AT=3, BT=6

1. FCFS

FCFS seperti antrian di kasir supermarket - siapa yang datang duluan, dia yang dilayani duluan.

Data Set 1

```
asus@LAPTOP-01JEN10A:~/Praktikum4/fcfs$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== HASIL PENJADWALAN FCFS ===
PID    AT    BT    CT    TAT    WT
-----
1       0     10    10     10     0
2       1     5     15     14     9
3       2     8     23     21    13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== GANTT CHART ===
| P1 | P2 | P3 |
0   10  15  23
```

Data Set 2

```
asus@LAPTOP-01JEN10A:~/Praktikum4/fcfs$ ./fcfs
=== SIMULASI ALGORITMA FCFS ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== HASIL PENJADWALAN FCFS ===
PID  AT  BT  CT  TAT  WT
-----
1    0   5   5   5   0
2    1   3   8   7   4
3    2   8  16  14   6
4    3   6  22  19  13

Average Turnaround Time: 11.25
Average Waiting Time: 5.75

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
0   5   8  16  22
```

2. SJF

SJF seperti kasir supermarket yang mendahulukan pembeli dengan belanjaan paling sedikit supaya antrian cepat berkurang.

Data Set 1

```
asus@LAPTOP-01JEN10A:~/Praktikum4/sjf$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== HASIL PENJADWALAN SJF ===
PID  AT  BT  CT  TAT  WT
---  --  --  --  ---  --
P1    0  10  10  10   0
P2    1   5  15  14   9
P3    2   8  23  21  13

Average Turnaround Time: 15.00
Average Waiting Time: 7.33

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P3
```

Data Set 2

```

asus@LAPTOP-01JEN10A:~/Praktikum4/sjf$ ./sjf
=== SIMULASI ALGORITMA SJF (Non-Preemptive) ===
Masukkan jumlah proses: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== HASIL PENJADWALAN SJF ===
PID  AT  BT  CT  TAT  WT
---  --  --  --  ---  --
P1    0   5   5   5   0
P2    1   3   8   7   4
P3    2   8  22  20  12
P4    3   6  14  11   5

Average Turnaround Time: 10.75
Average Waiting Time: 5.25

=== URUTAN EKSEKUSI ===
Proses dieksekusi dalam urutan: P1 -> P2 -> P4 -> P3

```

3. Round Robin

Round Robin seperti bermain game dengan teman - setiap orang main bergantian dengan waktu yang sama, misalnya 5 menit per orang.

Data Set 1

```

asus@LAPTOP-01JEN10A:~/Praktikum4/round_robin$ ./round-robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 3
Masukkan time quantum: 3

Proses P1
Arrival Time: 0
Burst Time: 10

Proses P2
Arrival Time: 1
Burst Time: 5

Proses P3
Arrival Time: 2
Burst Time: 8

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 7
Waktu 3: Menjalankan P1 -> Sisa waktu: 4
Waktu 6: Menjalankan P2 -> Sisa waktu: 2
Waktu 9: Menjalankan P3 -> Sisa waktu: 5
Waktu 12: Menjalankan P1 -> Sisa waktu: 1
Waktu 15: Menjalankan P1 -> Selesai pada waktu 16
Waktu 16: Menjalankan P1 -> Selesai pada waktu 16
Waktu 16: Menjalankan P2 -> Selesai pada waktu 18

=== HASIL PENJADWALAN ROUND ROBIN ===
PID  AT  BT  CT  TAT  WT
---  --  --  --  ---  --
P1    0  10  16  16   6
P2    1   5  18  17  12
P3    2   8  11  11  12

Average Turnaround Time: 10.326.00
Average Waiting Time: -63326884.00

```

Data Set 2

```

asus@LAPTOP-D1JEN10A:~/Praktikum4/round_robin$ ./round-robin
=== SIMULASI ALGORITMA ROUND ROBIN ===
Masukkan jumlah proses: 4
Masukkan time quantum: 4

Proses P1
Arrival Time: 0
Burst Time: 5

Proses P2
Arrival Time: 1
Burst Time: 3

Proses P3
Arrival Time: 2
Burst Time: 8

Proses P4
Arrival Time: 3
Burst Time: 6

=== PROSES EKSEKUSI ===
Waktu 0: Menjalankan P1 -> Sisa waktu: 1
Waktu 4: Menjalankan P1 -> Selesai pada waktu 5
Waktu 5: Menjalankan P2 -> Selesai pada waktu 8
Waktu 8: Menjalankan P3 -> Sisa waktu: 4
Waktu 12: Menjalankan P4 -> Sisa waktu: 2
Waktu 16: Menjalankan P1 -> Selesai pada waktu 16
Waktu 16: Menjalankan P3 -> Selesai pada waktu 20

=== HASIL PENJADWALAN ROUND ROBIN ===
PID    AT    BT    CT    TAT    WT
---    --    --    --    ---    --
P1      0      5    16     16     11
P2      1      3      8      7      4
P3      2      8     20     18     10
P4      3      6    25377  -1610743199  32764

Average Turnaround Time: -402685792.00
Average Waiting Time: 8197.25

```

Kesimpulannya:

- Jika ingin cepat dan efisien gunakan SJF.
- Jika ingin urut, adil dan sederhana gunakan FCFS.
- Jika ingin adil untuk semua gunakan Round Robin.

BAB II

ANALISIS EXPERIMENT KETIGA ALGORITMA

Jawab pertanyaan berikut:

1. Algoritma mana yang memberikan Average Waiting Time terkecil?
2. Mengapa hasil algoritma berbeda-beda?
3. Kapan sebaiknya menggunakan algoritma FCFS?
4. Kapan sebaiknya menggunakan algoritma SJF?
5. Kapan sebaiknya menggunakan algoritma Round Robin?

Jawaban:

1. Untuk data set 1 algoritma yang memberikan average waiting time terkecil adalah SJF dan FCFS dengan waktu 7.33
Untuk data set 2 algoritma yang memberikan average waiting time terkecil adalah SJF dengan waktu 5.25
2. Hasilnya berbeda di karenakan setiap algoritma memiliki cara proses tersendiri contohnya sjf focus pada waktu yang paling singkat terlebih dahulu, fcfs proses berdasarkan urutan masuknya, dan round robin focus membagi waktu proses dengan adil.
3. Gunakan FCFS jika system sederhana, tidak interaktif dan prosesnya bergantian.
4. Gunakan SJF Ketika system dapat mengira2 waktu eksekusi yang tepat dan ingin mencapai efisiensi maksimal.
5. Gunakan round robin saat system membutuhkan respons yang cepat, keadilan antar proses2, dan interaksi pengguna yang berkelanjutan.

BAB III

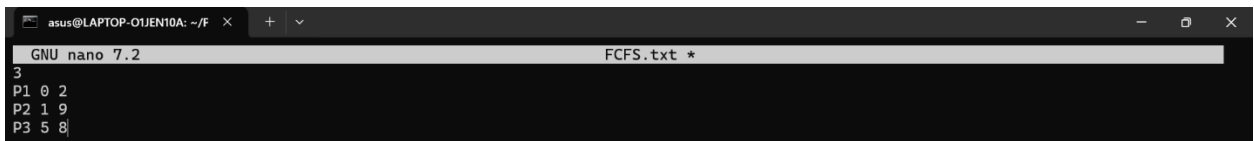
MODIFIKASI PROGRAM

Tambahkan fitur berikut pada salah satu program:

1. Tampilkan Gantt Chart yang lebih detail
2. Simpan hasil ke file text
3. Baca input dari file

1. FCFS

Buat file FCFS.txt



```
GNU nano 7.2 FCFS.txt *
3
P1 0 2
P2 1 9
P3 5 8
```

Perbaiki kode pada FCFS



```
GNU nano 7.2 modifikasi_fcfs.c
#include <stdio.h>
#include <stdlib.h>

struct Process {
    char id[5];
    int arrival, burst;
    int waiting, turnaround, completion;
};

int main() {
    FILE *fin, *fout;
    fin = fopen("FCFS.txt", "r");
    fout = fopen("Hasil_fcfs.txt", "w");

    if (fin == NULL) {
        printf("Gagal membuka file FCFS.txt!\n");
        return 1;
    }

    int n;
    fscanf(fin, "%d", &n);

    struct Process p[n];

    for (int i = 0; i < n; i++) {
        fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
    }

    fclose(fin);
}
```

```
asus@LAPTOP-OJEN10A: ~/F x + v
GNU nano 7.2 modifikasi_fcfs.c
// FCFS Scheduling
int current_time = 0;
float total_wt = 0, total_tat = 0;

printf("\n=== Hasil Penjadwalan FCFS ===\n");
printf("ID\tAT\tBT\tCT\tTAT\tWT\n");

fprintf(fout, "=== Hasil Penjadwalan FCFS ===\n");
fprintf(fout, "ID\tAT\tBT\tCT\tTAT\tWT\n");

for (int i = 0; i < n; i++) {
    if (current_time < p[i].arrival)
        current_time = p[i].arrival;

    p[i].completion = current_time + p[i].burst;
    p[i].turnaround = p[i].completion - p[i].arrival;
    p[i].waiting = p[i].turnaround - p[i].burst;

    total_wt += p[i].waiting;
    total_tat += p[i].turnaround;

    current_time = p[i].completion;

    printf("%s\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].arrival, p[i].burst,
        p[i].completion, p[i].turnaround, p[i].waiting);

    fprintf(fout, "%s\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].arrival, p[i].burst,
        p[i].completion, p[i].turnaround, p[i].waiting);
}

float avg_wt = total_wt / n;
float avg_tat = total_tat / n;

printf("\nRata-rata Waiting Time : %.2f\n", avg_wt);
printf("Rata-rata Turnaround Time: %.2f\n", avg_tat);

fprintf(fout, "\nRata-rata Waiting Time : %.2f\n", avg_wt);
fprintf(fout, "Rata-rata Turnaround Time: %.2f\n", avg_tat);

// Gantt Chart
printf("\n=== GANTT CHART ===\n ");
fprintf(fout, "\n=== GANTT CHART ===\n ");
for (int i = 0; i < n; i++) {
    printf("-----");
    fprintf(fout, "-----");
}
printf("\n|");
fprintf(fout, "\n|");
for (int i = 0; i < n; i++) {
    printf(" %s |", p[i].id);
    fprintf(fout, " %s |", p[i].id);
}
printf("\n ");
fprintf(fout, "\n ");
for (int i = 0; i < n; i++) {
    printf("-----");
    fprintf(fout, "-----");
}
printf("\n0");
fprintf(fout, "\n0");

for (int i = 0; i < n; i++) {
    printf(" %d", p[i].completion);
    fprintf(fout, " %d", p[i].completion);
}
printf("\n");
fprintf(fout, "\n");

fclose(fout);
return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo     M-6 Copy
```

Penjelasannya:

- `Current_time` digunakan untuk mencatat waktu.
- `CT (Completion Time)` = waktu selesai
- `TAT (Turnaround Time)` = `CT - AT`

- WT (Waiting Time) = $TAT - BT$
- Fin digunakan untuk membuka file input yaitu `FCFS.txt` yang berisikan proses.
- $Fout$ digunakan untuk membuat file output yaitu `Hasil_fcfs.txt` untuk menyimpan hasil outputnya.

Outputnya:

```
asus@LAPTOP-01JEN10A:~/Praktikum4/fcfs$ ./modifikasi_fcfs

=== Hasil Penjadwalan FCFS ===
ID   AT   BT   CT   TAT   WT
P1   0     2     2     2     0
P2   1     9    11    10     1
P3   5     8    19    14     6

Rata-rata Waiting Time : 2.33
Rata-rata Turnaround Time: 8.67

=== GANTT CHART ===
-----
| P1 | P2 | P3 |
-----
0     2     11    19
```

2. SJF

Buat file `SJF.txt`

```
asus@LAPTOP-01JEN10A: ~/F  X + v
GNU nano 7.2 SJF.txt
4
P1 1 3
P2 2 4
P3 3 5
P4 4 6
```

Perbaiki kode pada SJF

```
asus@LAPTOP-01JEN10A: ~/F  X + v
GNU nano 7.2 modifikasi_sjf.c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct Process {
    char id[5];
    int arrival, burst;
    int waiting, turnaround, completion;
    bool done;
};

int main() {
    FILE *fin = fopen("SJF.txt", "r");
    FILE *fout = fopen("hasil_sjf.txt", "w");

    if (fin == NULL) {
        printf("Gagal membuka file SJF.txt!\n");
        return 1;
    }

    int n;
    fscanf(fin, "%d", &n);

    struct Process p[n];

    for (int i = 0; i < n; i++) {
        fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
        p[i].done = false;
    }

    [ Read 119 lines ]
```

```
asus@LAPTOP-O1JEN10A: ~/F × + v
GNU nano 7.2 modifikasi_sjf.c
fclose(fin);

int completed = 0, current_time = 0;
float total_wt = 0, total_tat = 0;

printf("\n=== Hasil Penjadwalan SJF ===\n");
printf("ID\tAT\tBT\tCT\tTAT\tWT\n");

fprintf(fout, "=== Hasil Penjadwalan SJF ===\n");
fprintf(fout, "ID\tAT\tBT\tCT\tTAT\tWT\n");

while (completed < n) {
    int idx = -1, min_bt = 9999;

    // Cari proses dengan burst time terkecil yang sudah datang
    for (int i = 0; i < n; i++) {
        if (!p[i].done && p[i].arrival <= current_time && p[i].burst < min_bt) {
            min_bt = p[i].burst;
            idx = i;
        }
    }

    // Jika tidak ada proses yang datang, maju ke waktu berikutnya
    if (idx == -1) {
        current_time++;
        continue;
    }

    // Hitung waktu eksekusi proses yang dipilih
```

```
asus@LAPTOP-O1JEN10A: ~/F × + v
GNU nano 7.2 modifikasi_sjf.c
p[idx].completion = current_time + p[idx].burst;
p[idx].turnaround = p[idx].completion - p[idx].arrival;
p[idx].waiting = p[idx].turnaround - p[idx].burst;

total_wt += p[idx].waiting;
total_tat += p[idx].turnaround;

current_time = p[idx].completion;
p[idx].done = true;
completed++;

printf("%s\t%d\t%d\t%d\t%d\t%d\n",
        p[idx].id, p[idx].arrival, p[idx].burst,
        p[idx].completion, p[idx].turnaround, p[idx].waiting);

fprintf(fout, "%s\t%d\t%d\t%d\t%d\t%d\n",
        p[idx].id, p[idx].arrival, p[idx].burst,
        p[idx].completion, p[idx].turnaround, p[idx].waiting);
}

float avg_wt = total_wt / n;
float avg_tat = total_tat / n;

printf("\nRata-rata Waiting Time : %.2f\n", avg_wt);
printf("Rata-rata Turnaround Time: %.2f\n", avg_tat);

fprintf(fout, "\nRata-rata Waiting Time : %.2f\n", avg_wt);
fprintf(fout, "Rata-rata Turnaround Time: %.2f\n", avg_tat);
```

```

asus@LAPTOP-01JEN10A: ~/F  x  +  v
GNU nano 7.2                                modifikasi_sjf.c
// Gantt Chart
printf("\n=== GANTT CHART ===\n ");
fprintf(fout, "\n=== GANTT CHART ===\n ");
for (int i = 0; i < n; i++) {
    printf("-----");
    fprintf(fout, "-----");
}
printf("\n|");
fprintf(fout, "\n|");
for (int i = 0; i < n; i++) {
    printf(" %s |", p[i].id);
    fprintf(fout, " %s |", p[i].id);
}
printf("\n");
fprintf(fout, "\n ");
for (int i = 0; i < n; i++) {
    printf("-----");
    fprintf(fout, "-----");
}
printf("\n0");
fprintf(fout, "\n0");
for (int i = 0; i < n; i++) {
    printf(" %d", p[i].completion);
    fprintf(fout, " %d", p[i].completion);
}
printf("\n");
fprintf(fout, "\n");

fclose(fout);

return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   ^M-U Undo     ^M-A Set Mark
^X Exit      ^R Read File  ^N Replace    ^U Paste      ^J Justify    ^_ Go To Line  ^M-E Redo     ^M-G Copy

```

Penjelasannya:

- Completion Time (CT) waktu proses selesai.
- Turnaround Time (TAT) total waktu proses dalam system. ($TAT = CT - AT$)
- Waiting Time (WT) waktu proses menunggu giliran. ($WT = TAT - BT$)
- Completed jumlah proses yang selesai dilakukan.

Outputnya:

```

asus@LAPTOP-01JEN10A: ~/Praktikum4/sjf$ ./modifikasi_sjf

=== Hasil Penjadwalan SJF ===
ID  AT   BT   CT   TAT  WT
P1   1    3    4    3    0
P2   2    4    8    6    2
P3   3    5   13   10    5
P4   4    6   19   15    9

Rata-rata Waiting Time : 4.00
Rata-rata Turnaround Time: 8.50

=== GANTT CHART ===
| P1 | P2 | P3 | P4 |
-----
0    4    8   13   19

```

3. Round Robin

Buat file Round_Robin.txt

```

asus@LAPTOP-01JEN10A: ~/F  x  +  v
GNU nano 7.2                                Round_Robin.txt
3
P1 4 6
P2 1 2
P3 8 9

```

Perbaiki kode pada Round_Robin

```
asus@LAPTOP-OIJEN10A: ~/F  +  v
GNU nano 7.2 modifikasi_Round_Robin.c
#include <stdio.h>
#include <stdlib.h>

struct Process {
    char id[5];
    int arrival, burst, remaining;
    int waiting, turnaround, completion;
};

int main() {
    FILE *fin = fopen("Round_Robin.txt", "r");
    FILE *fout = fopen("hasil_rr.txt", "w");

    if (fin == NULL) {
        printf("Gagal membuka file Round_Robin.txt!\n");
        return 1;
    }

    int n, quantum = 3;
    fscanf(fin, "%d", &n);

    struct Process p[n];
    for (int i = 0; i < n; i++) {
        fscanf(fin, "%s %d %d", p[i].id, &p[i].arrival, &p[i].burst);
        p[i].remaining = p[i].burst;
    }
    fclose(fin);

    int completed = 0, current_time = 0;

    [ Read 112 lines ]

```

```
asus@LAPTOP-OIJEN10A: ~/F  +  v
GNU nano 7.2 modifikasi_Round_Robin.c
float total_wt = 0, total_tat = 0;
int done[n];
for (int i = 0; i < n; i++) done[i] = 0;

printf("\n=== Hasil Penjadwalan Round Robin (Q = %d) ===\n", quantum);
printf("ID\tAT\tBT\tCT\tTAT\tWT\n");

fprintf(fout, "=== Hasil Penjadwalan Round Robin (Q = %d) ===\n", quantum);
fprintf(fout, "ID\tAT\tBT\tCT\tTAT\tWT\n");

while (completed < n) {
    int progress = 0;
    for (int i = 0; i < n; i++) {
        if (p[i].arrival <= current_time && p[i].remaining > 0) {
            progress = 1;
            if (p[i].remaining > quantum) {
                current_time += quantum;
                p[i].remaining -= quantum;
            } else {
                current_time += p[i].remaining;
                p[i].remaining = 0;
                p[i].completion = current_time;
                p[i].turnaround = p[i].completion - p[i].arrival;
                p[i].waiting = p[i].turnaround - p[i].burst;
                total_tat += p[i].turnaround;
                total_wt += p[i].waiting;
                completed++;
            }
        }
    }
}

```

```
asus@LAPTOP-O1JEN10A: ~/F x + v
GNU nano 7.2 modifikasi_Round_Robin.c
    }
    if (!progress)
        current_time++;
}

for (int i = 0; i < n; i++) {
    printf("%s\t%d\t%d\t%d\t%d\t%d\n",
           p[i].id, p[i].arrival, p[i].burst,
           p[i].completion, p[i].turnaround, p[i].waiting);
    fprintf(fout, "%s\t%d\t%d\t%d\t%d\t%d\n",
            p[i].id, p[i].arrival, p[i].burst,
            p[i].completion, p[i].turnaround, p[i].waiting);
}

float avg_wt = total_wt / n;
float avg_tat = total_tat / n;

printf("\nRata-rata Waiting Time : %.2f\n", avg_wt);
printf("Rata-rata Turnaround Time: %.2f\n", avg_tat);

fprintf(fout, "\nRata-rata Waiting Time : %.2f\n", avg_wt);
fprintf(fout, "Rata-rata Turnaround Time: %.2f\n", avg_tat);

// Gantt Chart
printf("\n=== GANTT CHART ===\n ");
fprintf(fout, "\n=== GANTT CHART ===\n ");
for (int i = 0; i < n; i++) {
    printf("-----");
    fprintf(fout, "-----");
}

printf("\n");
fprintf(fout, "\n");
for (int i = 0; i < n; i++) {
    printf("  %s |", p[i].id);
    fprintf(fout, "  %s |", p[i].id);
}

printf("\n ");
fprintf(fout, "\n ");
for (int i = 0; i < n; i++) {
    printf("-----");
    fprintf(fout, "-----");
}

printf("\n0");
fprintf(fout, "\n0");
for (int i = 0; i < n; i++) {
    printf("      %d", p[i].completion);
    fprintf(fout, "      %d", p[i].completion);
}

printf("\n");
fprintf(fout, "\n");

fclose(fout);
return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo     M-C Copy
```

Penjelasannya:

- Remaining Adalah sisa waktu eksekusi.
- Int n, quantum = 3; mendefinisikan quantum yang berjumlah 3 / yang lain.
- Done[] digunakan untuk penanda apakah prosesnya sudah selesai atau belum.
- Arrival <= current time : proses sudah datang.
- Remaining > 0 : proses masih memiliki waktu yang tersisa.

Outputnya:

```
asus@LAPTOP-01JEN10A:~/Praktikum4/round_robin$ ./modifikasi_Round_Robin
```

```
=== Hasil Penjadwalan Round Robin (Q = 3) ===
```

ID	AT	BT	CT	TAT	WT
P1	4	6	10	6	0
P2	1	2	3	2	0
P3	8	9	19	11	2

Rata-rata Waiting Time : 0.67

Rata-rata Turnaround Time: 6.33

```
=== GANTT CHART ===
```

```
-----  
| P1 | P2 | P3 |  
-----  
0      10      3      19
```

BAB IV

EXPERIMENT DENGAN DATA BERBEDA UNTUK PERBANDINGAN ALGORITMA

Test Case 1: Proses dengan burst time bervariasi

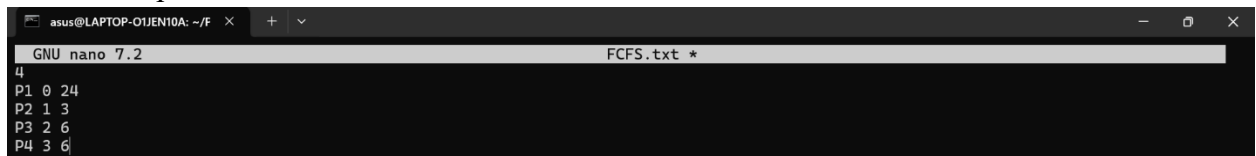
Jumlah proses: 4

Time quantum: 3

- P1: AT=0, BT=24
- P2: AT=1, BT=3
- P3: AT=2, BT=6
- P4: AT=3, BT=6

1. FCFS

Ganti kode pada FCFS.txt



```
GNU nano 7.2 FCFS.txt *
4
P1 0 24
P2 1 3
P3 2 6
P4 3 6
```

Output setelah dirubah



```
asus@LAPTOP-01JEN10A:~/Praktikum4/fcfs$ ./modifikasi_fcfs

=== Hasil Penjadwalan FCFS ===
ID   AT   BT   CT   TAT   WT
P1   0   24   24   24    0
P2   1    3   27   26   23
P3   2    6   33   31   25
P4   3    6   39   36   30

Rata-rata Waiting Time : 19.50
Rata-rata Turnaround Time: 29.25

=== GANTT CHART ===
-----
| P1 | P2 | P3 | P4 |
-----
0    24    27    33    39
```

2. SJF

Ganti kode pada SJF.txt



```
GNU nano 7.2 SJF.txt *
4
P1 0 24
P2 1 3
P3 2 6
P4 3 6
```

Output setelah dirubah

```

asus@LAPTOP-01JEN10A:~/Praktikum4/sjf$ ./modifikasi_sjf

=== Hasil Penjadwalan SJF ===
ID   AT   BT   CT   TAT   WT
P1   0   24   24   24    0
P2   1    3   27   26   23
P3   2    6   33   31   25
P4   3    6   39   36   30

Rata-rata Waiting Time : 19.50
Rata-rata Turnaround Time: 29.25

=== GANTT CHART ===
-----
| P1 | P2 | P3 | P4 |
-----
0    24    27    33    39

```

3. Round Robin

Ganti kode pada Round_Robin.txt

```

asus@LAPTOP-01JEN10A: ~/F  X + v
GNU nano 7.2 Round_Robin.txt *
4
P1 0 24
P2 1 3
P3 2 6
P4 3 6

```

Ubah jumlah Quantumnya

```

int n, quantum = 3;

```

Output setelah dirubah

```

asus@LAPTOP-01JEN10A:~/Praktikum4/round_robin$ ./modifikasi_Round_Robin

=== Hasil Penjadwalan Round Robin (Q = 3) ===
ID   AT   BT   CT   TAT   WT
P1   0   24   39   39   15
P2   1    3    6    5    2
P3   2    6   18   16   10
P4   3    6   21   18   12

Rata-rata Waiting Time : 9.75
Rata-rata Turnaround Time: 19.50

=== GANTT CHART ===
-----
| P1 | P2 | P3 | P4 |
-----
0    39    6    18    21

```

Test Case 2: Proses datang bersamaan

Jumlah proses: 5

Time quantum: 4

- P1: AT=0, BT=10
- P2: AT=0, BT=5
- P3: AT=0, BT=8
- P4: AT=0, BT=12
- P5: AT=0, BT=6

1. FCFS

Ganti kode pada FCFS.txt

```
asus@LAPTOP-O1JEN10A: ~/F x + v
GNU nano 7.2 FCFS.txt *
5
P1 0 10
P2 0 5
P3 0 8
P4 0 12
P5 0 6
```

Output setelah dirubah

```
asus@LAPTOP-O1JEN10A:~/Praktikum4/fcfs$ ./modifikasi_fcfs

=== Hasil Penjadwalan FCFS ===
ID  AT  BT  CT  TAT  WT
P1  0   10  10   10   0
P2  0    5   15   15  10
P3  0    8   23   23  15
P4  0   12   35   35  23
P5  0    6   41   41  35

Rata-rata Waiting Time : 16.60
Rata-rata Turnaround Time: 24.80

=== GANTT CHART ===
-----
| P1 | P2 | P3 | P4 | P5 |
-----
0    10   15   23   35   41
```

2. SJF

Ganti kode pada SJF.txt

```
asus@LAPTOP-O1JEN10A: ~/F x + v
GNU nano 7.2 SJF.txt *
5
P1 0 10
P2 0 5
P3 0 8
P4 0 12
P5 0 6
```

Output setelah dirubah

```
asus@LAPTOP-O1JEN10A:~/Praktikum4/sjf$ ./modifikasi_sjf

=== Hasil Penjadwalan SJF ===
ID  AT  BT  CT  TAT  WT
P2  0    5    5    5    0
P5  0    6   11   11    5
P3  0    8   19   19   11
P1  0   10   29   29   19
P4  0   12   41   41   29

Rata-rata Waiting Time : 12.80
Rata-rata Turnaround Time: 21.00

=== GANTT CHART ===
-----
| P1 | P2 | P3 | P4 | P5 |
-----
0   29    5   19   41   11
```

3. Round Robin

Ganti kode pada Round_Robin.txt

```
asus@LAPTOP-O1JEN10A: ~/F x + v
GNU nano 7.2 Round_Robin.txt *
5
P1 0 10
P2 0 5
P3 0 8
P4 0 12
P5 0 6
```

Ubah jumlah Quantumnya

```
int n, quantum = 4;
```

Output setelah dirubah

```
asus@LAPTOP-O1JEN10A:~/Praktikum4/round_robin$ ./modifikasi_Round_Robin

=== Hasil Penjadwalan Round Robin (Q = 4) ===
ID   AT   BT   CT   TAT  WT
P1   0    10   37   37   27
P2   0     5   25   25   20
P3   0     8   29   29   21
P4   0    12   41   41   29
P5   0     6   35   35   29

Rata-rata Waiting Time : 25.20
Rata-rata Turnaround Time: 33.40

=== GANTT CHART ===
-----
| P1 | P2 | P3 | P4 | P5 |
-----
0      37    25    29    41    35
```

Test Case 3: Proses datang dengan interval

Jumlah proses: 5

Time quantum: 2

- P1: AT=0, BT=8
- P2: AT=2, BT=4
- P3: AT=4, BT=9
- P4: AT=6, BT=5
- P5: AT=8, BT=3

1. FCFS

Ganti kode pada FCFS.txt

```
asus@LAPTOP-O1JEN10A: ~/F  X + v
GNU nano 7.2 FCFS.txt *
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3
```

Output setelah dirubah

```

asus@LAPTOP-01JEN10A:~/Praktikum4/fcfs$ ./modifikasi_fcfs

=== Hasil Penjadwalan FCFS ===
ID   AT   BT   CT   TAT   WT
P1   0     8    8    8     0
P2   2     4   12   10     6
P3   4     9   21   17     8
P4   6     5   26   20   15
P5   8     3   29   21   18

Rata-rata Waiting Time   : 9.40
Rata-rata Turnaround Time: 15.20

=== GANTT CHART ===
-----
| P1 | P2 | P3 | P4 | P5 |
-----
0     8    12    21    26    29

```

2. SJF

Ganti kode pada SJF.txt

```

asus@LAPTOP-01JEN10A: ~/F  x  +  v
GNU nano 7.2                               SJF.txt *
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3

```

Output setelah dirubah

```

asus@LAPTOP-01JEN10A:~/Praktikum4/sjf$ ./modifikasi_sjf

=== Hasil Penjadwalan SJF ===
ID   AT   BT   CT   TAT   WT
P1   0     8    8    8     0
P5   8     3   11    3     0
P2   2     4   15   13     9
P4   6     5   20   14     9
P3   4     9   29   25   16

Rata-rata Waiting Time   : 6.80
Rata-rata Turnaround Time: 12.60

=== GANTT CHART ===
-----
| P1 | P2 | P3 | P4 | P5 |
-----
0     8    15    29    20    11

```

3. Round Robin

Ganti kode pada Round_Robin.txt

```

asus@LAPTOP-01JEN10A: ~/F  x  +  v
GNU nano 7.2                               Round_Robin.txt *
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3

```

Ubah jumlah Quantumnya

```
int n, quantum = 2;
```

Output setelah dirubah

```
asus@LAPTOP-01JEN10A:~/Praktikum4/round_robin$ ./modifikasi_Round_Robin
```

```
=== Hasil Penjadwalan Round Robin (Q = 2) ===
```

ID	AT	BT	CT	TAT	WT
P1	0	8	26	26	18
P2	2	4	14	12	8
P3	4	9	29	25	16
P4	6	5	24	18	13
P5	8	3	19	11	8

```
Rata-rata Waiting Time : 12.60
```

```
Rata-rata Turnaround Time: 18.40
```

```
=== GANTT CHART ===
```

```
-----  
| P1 | P2 | P3 | P4 | P5 |  
-----  
0      26      14      29      24      19
```

BAB V

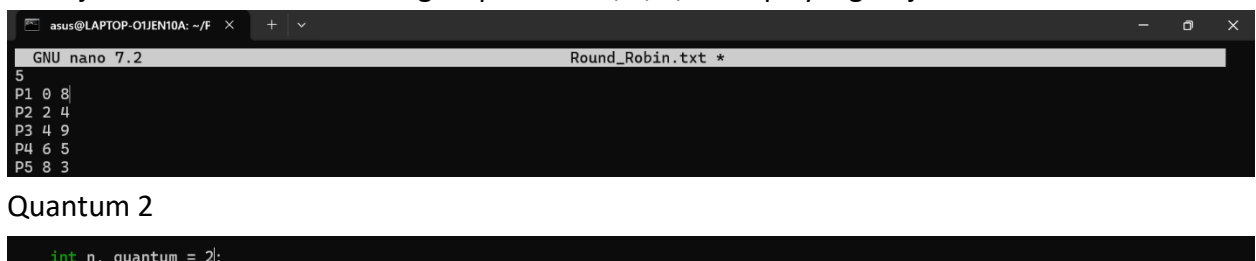
ANALISIS EXPERIMENT PERBANDINGAN ALGORITMA

1. Analisis Performa

- Dalam kondisi apa FCFS memberikan hasil terbaik?
FCFS berguna jika semua proses memiliki waktu yang sama dan datang secara berurutan. Contohnya antrian di supermarket.
- Mengapa SJF hampir selalu memberikan Average WT terkecil?
Karena sjf memprioritaskan proses yang memiliki waktu yang singkat terlebih dahulu oleh sebab itu dapat mengurangi waktu pemrosesan.
- Apa trade-off dari Round Robin?
Trade off pada round robin jika quantum kecil maka system dapat lebih responsive tetapi terjadi banyak context switching, sehingga overhead cpu meningkat, sebaliknya jika quantum besar overhead lebih kecil dan dampak negatifnya system kurang responsive.

2. Context Switching

- Algoritma mana yang paling banyak melakukan context switch?
Round robin dikarenakan setiap proses berjalan menggunakan quantum dan harus bergantian.
- Bagaimana pengaruh quantum terhadap jumlah context switch di Round Robin?
Semakin kecil quantum semakin banyak jumlah context switching yang akan terjadi, dan sebaliknya jika semakin besar quantum maka semakin kecil juga jumlah context switchingnya.
- Coba jalankan Round Robin dengan *quantum* 2, 4, 8, 16 - apa yang terjadi?



```
asus@LAPTOP-O1JEN10A: ~/F
GNU nano 7.2 Round_Robin.txt *
5
P1 0 8
P2 2 4
P3 4 9
P4 6 5
P5 8 3

Quantum 2

int n, quantum = 2;
```

```
asus@LAPTOP-01JEN10A:~/Praktikum4/round_robin$ ./modifikasi_Round_Robin
```

```
=== Hasil Penjadwalan Round Robin (Q = 2) ===
```

ID	AT	BT	CT	TAT	WT
P1	0	8	26	26	18
P2	2	4	14	12	8
P3	4	9	29	25	16
P4	6	5	24	18	13
P5	8	3	19	11	8

Rata-rata Waiting Time : 12.60
Rata-rata Turnaround Time: 18.40

```
=== GANTT CHART ===
```

```
-----  
| P1 | P2 | P3 | P4 | P5 |  
-----  
0      26      14      29      24      19
```

Quantum 4

```
int n, quantum = 4;
```

```
asus@LAPTOP-01JEN10A:~/Praktikum4/round_robin$ ./modifikasi_Round_Robin
```

```
=== Hasil Penjadwalan Round Robin (Q = 4) ===
```

ID	AT	BT	CT	TAT	WT
P1	0	8	23	23	15
P2	2	4	8	6	2
P3	4	9	29	25	16
P4	6	5	28	22	17
P5	8	3	19	11	8

Rata-rata Waiting Time : 11.60
Rata-rata Turnaround Time: 17.40

```
=== GANTT CHART ===
```

```
-----  
| P1 | P2 | P3 | P4 | P5 |  
-----  
0      23      8      29      28      19
```

Quantum 8

```
int n, quantum = 8;  
fscanf(fin, "%d", &n);
```

```
asus@LAPTOP-01JEN10A:~/Praktikum4/round_robin$ ./modifikasi_Round_Robin
```

```
=== Hasil Penjadwalan Round Robin (Q = 8) ===
```

ID	AT	BT	CT	TAT	WT
P1	0	8	8	8	0
P2	2	4	12	10	6
P3	4	9	29	25	16
P4	6	5	25	19	14
P5	8	3	28	20	17

Rata-rata Waiting Time : 10.60
Rata-rata Turnaround Time: 16.40

```
=== GANTT CHART ===
```

```
-----  
| P1 | P2 | P3 | P4 | P5 |  
-----  
0      8      12      29      25      28
```

Quantum 16

```
int n, quantum = 16;
```

```

asus@LAPTOP-01JEN10A:~/Praktikum4/round_robin$ ./modifikasi_Round_Robin

=== Hasil Penjadwalan Round Robin (Q = 16) ===
ID   AT   BT   CT   TAT  WT
P1   0    8    8    8    0
P2   2    4   12   10    6
P3   4    9   21   17    8
P4   6    5   26   20   15
P5   8    3   29   21   18

Rata-rata Waiting Time : 9.40
Rata-rata Turnaround Time: 15.20

=== GANTT CHART ===
-----
| P1 | P2 | P3 | P4 | P5 |
-----
0    8   12   21   26   29

```

Berdasarkan proses diatas benar semakin kecil quantum semakin banyak jumlah context switching yang akan terjadi, dan sebaliknya jika semakin besar quantum maka semakin kecil juga jumlah context switchingnya.

3. Fairness

- Algoritma mana yang paling "adil" untuk semua proses?
Round Robin karena setiap proses diberi kesempatan yang sama untuk berjalan. Sehingga tidak ada proses yang ditinggal atau didahulukan.
- Proses mana yang paling dirugikan di algoritma FCFS?
Proses yang datang belakangan karena FCFS melakukan proses secara urut dari yang awal datang hingga akhir.
- Apakah ada kemungkinan starvation di SJF?
Bisa kemungkinan adanya starvation, terutama untuk proses yang besar yang datang diawal karena SJF mendahulukan proses yang kecil terlebih dahulu.

4. Real World Application

- Untuk *web server* yang melayani banyak kecil, algoritma mana yang cocok?
Round robin dikarenakan memberikan respon cepat, pembagian cpu yang adil, dan performa yang stabil.
- Untuk *render video* (task besar), algoritma mana yang cocok?
Untuk render video besar dan lama disarankan menggunakan SJF jika waktu proses dapat diprediksi, jika waktu proses tidak diketahui maka bisa menggunakan FCFS.
- Untuk OS *desktop* (interaktif), algoritma mana yang cocok?
Gunakan Round Robin karena memberikan respons cepat, keadilan antar-prosesnya dan menjaga kelancaran pada multitaskingnya.